

CoreBuster

Project Overview — Group Briefing

COMP307 · Polished for the 19 August meeting

This document is a full catch-up on the project idea so everyone's on the same page before splitting up work. It covers what we're making, why it's designed the way it is, how it plays end to end — including the economy and wave structure — and how it gets built in Unity. Two companion documents go with this one: the Game Concept and Overview (submitted 21 August) and the Team Getting Started Guide (the step-by-step build plan, since none of us have used Unity before).

1. What we're making

CoreBuster is a first-person, wave-based shooter built for the COMP307 project brief. You fight through escalating waves of robotic enemies in a single arena. Every enemy is assembled from four modular parts — head, body, arms, legs — and each part changes that enemy's stats and behaviour, so no two enemies play quite the same.

You have two tools: a primary weapon for standard damage, and the Disassembler — your ultimate ability. Firing the Disassembler at an enemy sends it into a vulnerable, frozen state, seizing up like it's being electrocuted. While frozen, you can move in and execute it by pulling out its core — an instant kill and an extra reward. Because the Disassembler is an ultimate rather than an always-available tool, using it well — picking the right target and the right moment — matters.

2. Why this idea

- Directly demonstrates what the module assesses: transformation (parts parented to sockets), modelling (custom parts and core, satisfying the first-principles requirement), and rendering (a custom shader on the core).
- One clear, describable hook — easy to explain and demo in a short presentation.
- Scoped tightly on purpose, since this project is worth 10%: a single fixed arena, wave-based rather than fully procedural spawning, and a small curated set of module variants rather than fully randomised stats.

3. How it plays

- You enter a single arena and survive a sequence of waves; enemy count and difficulty increase each round.
- Between waves, a short breather lets you spend currency on upgrades.
- Fire your primary weapon as your main source of damage.
- Fire the Disassembler — your ultimate, limited by a cooldown or charge — to freeze an enemy in a vulnerable, electrocuted state.
- Move in on a frozen enemy and execute it by pulling out its core, for a guaranteed kill and an extra reward.

- Leave a frozen enemy too long without executing it and it can recover, so timing matters.
- Loss: your health hits zero. Win: survive a set number of waves and defeat a final boss built from high-tier parts.

Played entirely first-person — mouse controls both looking and aiming, matching the games most of us already play, giving a familiar reference point while we're all learning Unity from scratch.

4. The enemies — modular by design

Each enemy is built at spawn from a small, curated set of variants per slot (not fully randomised, to keep this manageable while learning). Even two options per slot already gives good visible variety:

Slot	Example variants	Effect on the enemy
Body	Armoured / Light	Armoured = more health, slower · Light = less health, faster
Legs	Quick / Standard	Quick legs = increased movement speed
Arms	Melee / Ranged	Determines attack type and effective range
Head	Sensor / Reinforced	Affects detection range or a weakpoint behaviour

Under the hood, each variant is a small data asset (a Unity ScriptableObject) holding a mesh and a stat modifier. A script picks one variant per slot at spawn and slots the part into the enemy's skeleton — no enemy is hand-placed; the combination is what creates the variety.

5. The signature mechanic — the Disassembler ultimate, freeze, and the galaxy shader

This is the centrepiece of the game and where most of the presentation's “wow factor” comes from, in three stages:

- Freeze: firing the Disassembler ultimate at an enemy stops its AI cold and locks it into a vulnerable state — seizing and juddering in place like it's being electrocuted. A clear, readable signal that the enemy is now open to execution.
- Core exposed: while frozen, a core hidden inside the body becomes visible and vulnerable. It uses a custom-built “galaxy” shader — a dark surface with slowly shifting purple/blue/teal colour patches, sparse twinkling star highlights, and a glowing rim — unmistakable as the thing to go for. This is the required custom-shader deliverable.
- Execute: moving in on a frozen enemy and pulling out its core kills it instantly and drops an extra reward on top of its normal drop. Parts then fall to the ground as pickups — and a hazard, since an uncollected part can be scavenged by another enemy. Wait too long and the freeze wears off, so timing the ultimate well matters.

6. Economy & upgrades

Executing a core drops Cores — the run's single currency, and the direct source of the game's name. One currency type only; splitting into multiple resources adds bookkeeping the scope doesn't need.

- Base drop: every executed enemy drops a consistent baseline amount.
- Tier bonus: enemies built from higher-tier parts drop more — rewards recognising part tiers on sight.
- Scavenge trade-off: letting an enemy scavenge dropped parts makes it stronger, but it drops more Cores when eventually killed — risk vs. reward instead of a pure penalty.

- Cores only spend between waves, not mid-fight, keeping the breather meaningful.

Between waves, a shop offers three simple upgrade tracks, matching the three tools you actually use:

Track	What it does	Tiers (suggested effects)
Armor	Reduces incoming damage / raises max HP	T1: +20% max HP · T2: +15% damage reduction · T3: minor HP regen out of combat
Primary Weapon	Your day-to-day damage dealer	T1: +damage · T2: +fire rate · T3: +magazine size / faster reload
Disassembler	Power of the ultimate ability	T1: shorter cooldown/faster recharge · T2: increased range · T3: enemies stay frozen longer before recovering

Costs scale up per tier (roughly ×1.8) so early choices feel meaningful. Breather timer: 20–30 seconds, with an early-skip option.

7. Wave structure & the arena

- 8 regular waves + 1 boss wave — a round, easy-to-balance number for the time available.
- Scaling: enemy count increases roughly linearly per wave; part-variant tiers unlock gradually — waves 1–3 base variants only, 4–6 introduce tier two, 7–8 mix freely.
- Spawning: 2–4 fixed perimeter spawn points, staggered through the wave rather than all at once — keeps NavMesh pathing predictable for a first Unity project.
- Boss wave: a single high-tier boss (all four slots top-tier, more HP, an extra attack pattern) rather than a boss plus minions — simpler to script and still reads as a climax.
- The arena: one fixed arena with light verticality and cover (a few raised platforms, low walls, a central obstacle) so the Disassembler mechanic has room to matter — ducking behind cover while a core is still exposed creates real tension. Geometry stays simple (boxes, ramps, cylinders); the first-principles modelling requirement is met by the enemy parts and core, not the arena.

8. Win/loss and progression

- Loss: player health reaches zero at any point — the run ends.
- Win: survive all waves and defeat the final boss.
- Currency earned from executions is spent on upgrades between waves, giving a build-progression feel typical of the genre even within one arena.

9. What we're building it in

Piece	Approach
Engine	Unity, Universal Render Pipeline (URP) — needed for the shader work.
Player controller	Unity's official Starter Assets first-person controller, rather than custom movement/mouse-look.
Enemy AI	Unity's built-in NavMesh system for chase behaviour, instead of custom pathfinding.

Modular parts	ScriptableObject data assets per part variant, assembled onto socket transforms at spawn.
Core shader	Shader Graph (visual, node-based) for the galaxy effect — no hand-written shader code.
Source control	A shared GitHub repository, as required, with a Unity-specific .gitignore.

10. Meeting the project brief

- One or more playable characters that move and interact with the environment — the first-person player.
- NPCs with defined behaviour — the modular enemies and their chase AI.
- Win and loss conditions — wave survival / boss defeat vs. player health reaching zero.
- Assets modelled from first principles — the modular parts and core are custom-modelled, not store-bought.
- At least one custom shader demonstrating a graphics technique — the galaxy/core shader.
- Source code submitted for reproducibility — the GitHub repository.
- Completed in a group of 4 — us.

11. Visual identity — worn machinery outside, impossible cosmos inside

The name, logo, and enemy look all share one design principle, carried straight through from the core shader spec: exteriors read as ordinary, weathered hardware; the only cosmic, glowing material in the whole game is the core, and it's hidden until an enemy is frozen. That contrast is what makes the reveal moment land.



- Name — CoreBuster: chosen over Scrapfall for naming the action rather than the setting. It reads clearly in a short pitch, has arcade/action-game energy, and lines up directly with the game's currency (Cores) and its central mechanic (pulling the core to execute an enemy).
- Palette — brass and gunmetal exteriors, with rust as an accent; the core itself sits in purple/blue/teal against void black, matching the Shader Graph spec in Section 5.
- Exteriors (enemies, arena): riveted plating, exposed pistons, patina and scorch marks. Reads as salvaged and mechanical, not sleek sci-fi — this is the “Treasure Planet” reference point for the group.
- Interiors (the core): everything closes down except the exposed core — deep space colour, drifting nebula patches, faint star twinkle, a glowing rim. Only visible while an enemy is frozen, so it stays a reveal rather than a constant visual.
- Logo: a riveted brass/gunmetal ring (the mechanical exterior) around an exposed core burst — white-hot centre, blue-to-purple nebula glow, faint scattered stars — echoing the shader spec directly. Delivered as a vector SVG so it scales cleanly for slides, the GitHub README, or a title screen.

Open item from the enemy concept mockup: the current sketch textures the whole body as cosmic rather than isolating the core, which loses the “nothing to expose” reveal moment the brief calls for. Worth settling before this becomes the actual in-engine shader — either isolate the core treatment to just the core, or confirm the group wants a fuller cosmic-body look instead.

12. Timeline

Deliverable	Due	What's needed
Game Concept and Overview	Fri 21 Aug 2026	Document drafted — needs group review and names/student numbers added.
First Prototype	Fri 18 Sep 2026	10-minute presentation with a working WIP demo, remaining-work timeline, and challenges.
Final Deliverable	Wed 14 Oct 2026	Presentation, final source code + README, slides, self/buddy evaluation.

13. Before / at the 19th

☐ Everyone reads this document and skims the Team Guide before the meeting.

☐ Game name is settled: CoreBuster — confirm the group is happy to lock it in at the meeting.

☐ Everyone installs Unity Hub and the same LTS version, and gets access to the shared GitHub repo.

☐ Confirm names and student numbers for the group email — send immediately if it hasn't gone out.

☐ At the meeting: walk through this overview, agree the rough work split, and get everyone through Phase 0 before splitting up.

Internal working document — not for submission.